Chapter 2 — The Transformer Architecture Deep Dive

> *"The transformer architecture is not just the model you optimize. It is the workload specification that determines what your hardware must do. Understand its shapes, its memory access patterns, and where compute goes — and every hardware decision downstream becomes predictable."*

Chapter Overview

This chapter is the workload specification for the rest of the book. Every performance optimization technique in Chapters 6–17 is a technique applied to the operations described here. The chapter derives the attention mechanism from first principles, counts FLOPs and bytes for every operation in a real model (LLaMA-3 70B), introduces the attention variants that define modern LLM architecture, and closes with the scaling laws that drive every infrastructure investment decision at major AI labs.

Sections in this chapter:

- 2.1 Why Transformers Won — and Why That Matters for Performance
- 2.2 The Transformer Block — Anatomy of Every Modern LLM
- 2.3 Scaled Dot-Product Attention — Full Derivation and Complexity
- 2.4 Multi-Head Attention — What Multiple Heads Actually Do
- 2.5 Attention Variants — MHA → MQA → GQA → MLA (KV Cache Memory Math)
- 2.6 Feed-Forward Networks — SwiGLU, Expansion Ratios, and FLOP Dominance
- 2.7 Positional Encoding — Sinusoidal, ALiBi, and RoPE
- 2.8 RoPE — Full Derivation and Why It Enables Long Contexts
- 2.9 Normalization — LayerNorm, RMSNorm, Pre-Norm vs Post-Norm [r5]
- 2.10 The Complete FLOP Budget — LLaMA-3 70B Worked Example
- 2.11 Encoder, Decoder, and Encoder-Decoder Architectures
- 2.12 Scaling Laws — What Performance Engineers Must Know
- 2.13 Modern Architecture Comparison
- 2.14 The Performance Engineering Map
- 2.15 Key Takeaways and Review Questions

2.1 Why Transformers Won — and Why That Matters for Performance

The transformer architecture, introduced in "Attention Is All You Need" (Vaswani et al., 2017), displaced recurrent networks for language modeling and has since conquered vision, audio, biology, and robotics. Understanding *why* it won is not history — it directly explains the GPU bottlenecks you will spend your career optimizing.

```
TRANSFORMER VS RNN — PERFORMANCE ENGINEERING IMPLICATIONS:

  Property           | RNN / LSTM              | Transformer               | Perf
Implication
```

Property	RNN / LSTM	Transformer	Perf Implication
Token processing	Sequential (t requires t-1)	Parallel — all tokens processed simultaneously	Transformers saturate GPU Tensor Cores; RNNs do not
Long-range deps	Vanishes (gradient decay)	O(1) distance — any token attends to any	Enables 128K+ context; creates O(N^2) attention cost
Memory footprint	O(1) state per layer	O(N·d) KV cache per layer at inference	The KV cache problem — entire Ch 11 dedicated to this
GPU utilization	Poor — serial compute, low occupancy	Excellent — massive parallel GEMM operations	H100 at 70%+ MFU for large batch training
Scalability	Diminishing beyond ~1B parameters	Log-linear with params	Enables GPT-4, LLaMA-3 405B; scaling laws hold

The core insight: Transformers are parallelization-friendly at the cost of being memory-hungry. Every optimization in this book is a negotiation between compute parallelism and memory capacity/bandwidth. The transformer's success is inseparable from the HBM revolution in GPU architecture — and the HBM constraints are what created the field of LLM performance engineering.

2.2 The Transformer Block — Anatomy of Every Modern LLM

Every language model, regardless of size or vendor, is a stack of identical transformer blocks. One block contains exactly two sub-layers: a multi-head attention layer and a feed-forward network (MLP), each preceded by a normalization layer and connected by residual (skip) connections.

```
ONE TRANSFORMER BLOCK (Pre-Norm variant — LLaMA, GPT-3, all modern LLMs):

  x' = x + Attention( RMSNorm(x) )       <- residual + attention sublayer
  y  = x' + FFN( RMSNorm(x') )           <- residual + MLP sublayer

Full model:
  h_0 = Embedding(tokens)                <- input tokens -> dense vectors [vocab -
> d_model]
  h_l = TransformerBlock(h_{l-1})        for l in 1..L  (L stacked identical
blocks)
  logits = Unembed( RMSNorm(h_L) )       <- project back to vocab size [d_model ->
vocab]
  probs  = Softmax(logits)               <- token probability distribution

Data shapes through a LLaMA-3 70B forward pass (batch=1, seq=4096):
  Input tokens:       [1, 4096]                  (4096 token IDs)
  After embedding:    [1, 4096, 8192]            (4096 tokens, 8192-dim)
  After attention:    [1, 4096, 8192]            (same shape, attended)
  After FFN:          [1, 4096, 8192]            (same shape, transformed)
  Final logits:       [1, 4096, 128256]          (token probability over vocab)
```

2.2.1 Residual Connections — Why They Are Not Optional

```
RESIDUAL CONNECTION — GRADIENT FLOW MATH:

  Without residual: y = F(x)
    dy/dx = dF/dx   (can vanish to 0 in deep networks — gradient death)

  With residual:    y = x + F(x)
    dy/dx = 1 + dF/dx   (the "+1" guarantees non-vanishing gradient flow)

  Result at 80 layers (LLaMA-3 70B):
    Without residuals: gradient magnitude = prod(dF/dx at each layer)
                       At typical dF/dx ~ 0.9: 0.9^80 = 0.00024  -> vanished
    With residuals:    gradient always has at least 1 contribution from identity
                       Pre-Norm: residual is always well-scaled -> stable training
```

```
  Pre-Norm vs Post-Norm placement:
    Post-Norm (original 2017): y = LayerNorm(x + F(x))
      Problem: normalizing after residual can suppress gradient magnitude at
               large depth. Requires careful learning rate warm-up at scale.
      Used in: original Transformer, BERT, GPT-1.

    Pre-Norm (modern):  y = x + F(LayerNorm(x))
      Advantage: residual path bypasses normalization entirely.
                 Gradient flows through the raw residual with value exactly 1.
                 No learning rate warm-up required. Stable from step 1.
      Used in: GPT-2+, LLaMA 1/2/3, Mistral, Gemma, Falcon, all production LLMs.
```

2.3 Scaled Dot-Product Attention — Full Derivation and Complexity

Attention lets every output token position be a weighted sum of all input token representations, where the weights are dynamically computed from the content of the tokens.

2.3.1 The Attention Formula — Full Derivation

```
GOAL: For each query token, compute a weighted mixture of value vectors,
      where weights measure relevance between the query and each key.

STEP 1 — Project input X into three spaces:
  Q = X · W_Q    Q in R^(S x d_k)    "What am I looking for?"
  K = X · W_K    K in R^(S x d_k)    "What do I contain?"
  V = X · W_V    V in R^(S x d_v)    "What will I contribute if selected?"

  Where:
    X in R^(S x d_model)  — input sequence (S tokens, d_model dimensions)
    W_Q, W_K in R^(d_model x d_k)  — projection matrices
    W_V in R^(d_model x d_v)

STEP 2 — Compute raw attention scores:
  scores = Q · K^T    in R^(S x S)

  Why QK^T? Each entry scores[i,j] = Q[i] · K[j] measures the dot-product
  similarity between query i and key j.
  High value = query i "wants" information from position j.

STEP 3 — Scale by sqrt(d_k):
  scores_scaled = scores / sqrt(d_k)
```

```
  Why sqrt(d_k)? Without scaling, dot products grow with d_k (each component
  adds a contribution). At large d_k, raw dot products push softmax into
  near-zero gradient saturation. Dividing by sqrt(d_k) keeps variance O(1)
  regardless of head dimension.

  Proof: Var(Q_i · K_j) = d_k * Var(Q_ik) * Var(K_jk) (independent Gaussians)
         Scaling by 1/sqrt(d_k) restores variance to 1.

STEP 4 — Apply causal mask (decoder-only models):
  For position i, zero out scores[i,j] for all j > i (future tokens).
  Implementation: add -inf to future positions before softmax.

STEP 5 — Softmax normalization:
  weights = Softmax(scores_scaled)    in R^(S x S)
  weights[i] = exp(scores_scaled[i]) / sum(exp(scores_scaled[i]))

STEP 6 — Weighted sum of values:
  output = weights · V    in R^(S x d_v)

COMPLETE FORMULA:
  Attention(Q, K, V) = Softmax( Q·K^T / sqrt(d_k) + mask ) · V
```

2.3.2 Attention Complexity Analysis

```
ATTENTION COMPUTATIONAL COMPLEXITY:

  For sequence length S, head dimension d_k:

  Operation             | FLOPs             | Memory      | Notes
  ———————————————————————————————————————————————————————————————————————
  Q·K^T                 | 2·S^2·d_k         | O(S^2)      | Dominant at long S
  Softmax               | ~3·S^2            | O(S^2)      | exp, sum, divide
  Weights·V             | 2·S^2·d_v         | O(S^2)      | Symmetric to QK^T
  ———————————————————————————————————————————————————————————————————————
  Total attention score | 4·S^2·d_k + 3·S^2 | O(S^2)     | O(S^2) in both
  QKV projections       | 6·S·d_model·d_k   | O(S·d)      | O(S) — linear!

  KEY INSIGHT: QKV projections are O(S) — linear in sequence length.
               Attention score computation is O(S^2) — quadratic.

  At what sequence length does attention dominate over projections?
    Attention dominates when: 4·S^2·d_k > 6·S·d_model·d_k
```

```
    Simplified: S > 1.5 × d_model
    For LLaMA-3 70B (d_model=8192): attention dominates at S > 12,288

  Practical implication (LLaMA-3 70B):
    At S=2048:  attention = 25% of FLOPs (GEMM dominates)
    At S=8192:  attention = 50% of FLOPs (roughly equal)
    At S=32768: attention = 80% of FLOPs (attention dominates — costly)
    At S=128K:  attention = 97%+ (FlashAttention is not optional)
```

2.3.3 Concrete 3×3 Attention Worked Example

The attention formula is clean on paper. This worked example traces exact numbers through a minimal case so the mechanism becomes intuitive before scaling to production dimensions.

```
SETUP: 3-token sequence ["The", "cat", "sat"]
       Single attention head, d_k = d_v = 2 (toy dimensions for clarity)

STEP 1 — Q, K, V matrices (after projection, values shown directly):
  Q = | 1.0  0.5 |    K = | 1.0  0.5 |    V = | 1.0  0.0 |
      | 0.8  1.2 |        | 0.8  1.2 |        | 0.5  1.0 |
      | 0.2  0.9 |        | 0.2  0.9 |        | 0.2  0.8 |

STEP 2 — Raw scores: S = Q · K^T
  S[1,1] = 1.0×1.0 + 0.5×0.5 = 1.25    ("The" attends to "The")
  S[1,2] = 1.0×0.8 + 0.5×1.2 = 1.40    ("The" attends to "cat")
  S[1,3] = 1.0×0.2 + 0.5×0.9 = 0.65    ("The" attends to "sat")
  ... (complete S is 3×3)

  S = | 1.25  1.40  0.65 |
      | 1.40  1.72  0.94 |
      | 0.65  0.94  0.85 |

STEP 3 — Scale by sqrt(d_k) = sqrt(2) ≈ 1.414:
  S_scaled = S / 1.414
  S_scaled[1] = | 0.884  0.990  0.460 |

STEP 4 — Causal mask (token 1 can only attend to token 1, etc.):
  Apply -inf to upper triangle:
  S_masked[1] = | 0.884  -inf   -inf |
  S_masked[2] = | 0.990   1.216  -inf |
  S_masked[3] = | 0.460   0.665  0.601 |

STEP 5 — Softmax (after masking, -inf becomes 0):
  Row 1: [1.0,    0.0,   0.0]    ("The" attends only to itself)
  Row 2: [0.443,  0.557, 0.0]    ("cat" splits between itself and "The")
  Row 3: [0.286,  0.354, 0.360]  ("sat" splits roughly evenly — "sat" attended
most)

STEP 6 — Output: weights · V
  Output[1] = 1.0×[1.0, 0.0] + 0.0×[0.5, 1.0] + 0.0×[0.2, 0.8] = [1.0, 0.0]
  Output[2] = 0.443×[1.0,0.0] + 0.557×[0.5,1.0] = [0.722, 0.557]
  Output[3] = 0.286×[1.0,0.0]+0.354×[0.5,1.0]+0.360×[0.2,0.8]
            = [0.286+0.177+0.072, 0+0.354+0.288] = [0.535, 0.642]

INTERPRETATION:
  "The" (row 1): attends only to itself (causal mask blocks future)
  "cat" (row 2): blends "The" (44%) and "cat" (56%) representations
```

```
  "sat" (row 3): blends all three roughly equally — it can see the full context

PERFORMANCE NOTE:
  This 3×3 example required 3×3×2 = 18 multiply-adds for QK^T
  For LLaMA-3 70B at S=4096: QK^T = 4096×4096×128 per head × 64 heads
  = 138 billion multiply-adds just for attention scores per layer.
  FlashAttention eliminates storing this 4096×4096 matrix — see Ch 7.
```

2.4 Multi-Head Attention — What Multiple Heads Actually Do

```
MULTI-HEAD ATTENTION (MHA):

  Instead of one set of Q, K, V projections, run H parallel attention heads,
  each with its own projection matrices and its own learned attention patterns.

  For head h:
    Q_h = X · W_Q^h    W_Q^h in R^(d_model x d_head)
    K_h = X · W_K^h    W_K^h in R^(d_model x d_head)
    V_h = X · W_V^h    W_V^h in R^(d_model x d_head)

    head_h = Attention(Q_h, K_h, V_h)    in R^(S x d_head)

  Concatenate all heads and project back:
    MultiHead(X) = Concat(head_1, ..., head_H) · W_O
    W_O in R^(H*d_head x d_model)

DIMENSION CONVENTION (LLaMA-3 70B):
  d_model = 8,192    (full hidden dimension)
  H       = 64       (number of attention heads)
  d_head  = 128      (per-head dimension = d_model / H)
  H * d_head = 64 * 128 = 8,192  (matches d_model exactly — standard)

WHY MULTIPLE HEADS?
  Each head learns to attend to different aspects of the input:
    Head 1 might capture subject-verb relationships
    Head 2 might capture coreference (pronoun → noun)
    Head 8 might capture syntactic dependency
    Head 32 might capture semantic similarity
  No single head can specialize in all patterns simultaneously.
  Multi-head provides parallel diverse attention "queries".
```

```
PERFORMANCE NOTE:
  In practice, all H heads are computed together as a single batched GEMM:
    W_QKV = [W_Q; W_K; W_V]  in R^(d_model x 3*d_model)  (concatenated)
    QKV = X · W_QKV            in R^(S x 3*d_model)       (one GEMM)
  Then split and reshape for multi-head computation.
  This is why "QKV projection" appears as a single large GEMM in profilers.
```

2.5 Attention Variants — MHA → MQA → GQA → MLA

The evolution from Multi-Head Attention to Grouped-Query Attention is driven entirely by KV cache memory pressure at serving time. Understanding the math here is essential for every serving and inference engineer.

2.5.1 The KV Cache Memory Problem

```
KV CACHE SIZE — THE SCALING PROBLEM:

  For each token in context, each layer stores one K and one V vector.
  Per-layer KV cache size for one sequence:

    KV_bytes_per_layer = 2  (K and V) × S (tokens) × H_kv (KV heads) × d_head ×
bytes_per_element

  For LLaMA-3 70B MHA (H_kv = H_q = 64, BF16):
    Per layer = 2 × S × 64 × 128 × 2 bytes = 32,768 × S bytes
    80 layers = 80 × 32,768 × S = 2,621,440 × S bytes

    At S=4096:  KV cache = 2,621,440 × 4096 = 10.7 GB  (per sequence!)
    At S=128K:  KV cache = 336 GB  (impossible to fit — model is only 140 GB)

  This is why MHA cannot scale to long contexts without modification.
  GQA and MQA exist to solve this specific problem.
```

2.5.2 The Three Attention Variants

```
ATTENTION VARIANT COMPARISON:

  MHA (Multi-Head Attention) — original:
    H_q query heads,  H_q key heads,  H_q value heads
    Every query head has its own K and V head.
    LLaMA-2 7B, GPT-2, BERT.
```

```
  MQA (Multi-Query Attention) — Palm, Falcon:
    H_q query heads,  1 key head,  1 value head
    All query heads SHARE one K and one V projection.
    KV cache reduction: H_q × smaller = 64× smaller vs MHA for LLaMA-3 70B.
    Quality: ~1-2% degradation vs MHA on most benchmarks.

  GQA (Grouped-Query Attention) — LLaMA-3, Mistral, Gemma:
    H_q query heads,  G key heads,  G value heads   where 1 < G < H_q
    Query heads divided into G groups, each group shares one K and one V.
    LLaMA-3 70B: H_q = 64, G = H_kv = 8  (8 queries per KV group)

  DIMENSION TABLE FOR LLaMA-3 70B:
    Variant  | Q heads | KV heads | KV cache (4096 tokens) | Quality loss
    ──────────────────────────────────────────────────────────────────────
    MHA      |    64   |    64    |    10.7 GB/seq         | 0% (baseline)
    GQA      |    64   |     8    |    1.34 GB/seq         | ~0% (well-tuned)
    MQA      |    64   |     1    |    167 MB/seq          | 1-2%

  GQA MEMORY CALCULATION (LLaMA-3 70B):
    Per layer: 2 × S × 8 × 128 × 2 bytes = 4096 × S bytes
    80 layers: 80 × 4096 × S = 327,680 × S bytes
    At S=4096: 327,680 × 4096 = 1.34 GB  (vs 10.7 GB MHA = 8× reduction!)
    At S=128K: 41.9 GB  (fits! — this is how LLaMA-3 achieves 128K context)
```

2.5.3 MLA — Multi-Head Latent Attention (DeepSeek-V3)

```
MLA — The Next Generation (DeepSeek-V3, DeepSeek-R1):

  Problem with GQA: still caches full d_head × H_kv per token.
  MLA insight: compress the KV projection into a low-dimensional latent vector.

  Instead of caching K and V separately:
    c_KV = X · W_DKV    c_KV in R^(S x d_c)   where d_c << H_kv × d_head
    K = c_KV · W_UK     K in R^(S x H × d_head)   (up-projected at attention time)
    V = c_KV · W_UV     V in R^(S x H × d_head)   (up-projected at attention time)

  Cache ONLY c_KV (the compressed latent), not full K and V.

  DeepSeek-V3: d_c = 512  (vs 8 × 128 = 1024 for GQA with H_kv=8)
    Per-layer cache = S × 512 × 2 bytes
    At S=4096, 61 layers: 61 × 4096 × 512 × 2 = 256 MB  (tiny!)
```

```
  KV cache vs GQA at equal model quality:
    GQA (LLaMA-3 70B style):  1.34 GB at S=4096
    MLA (DeepSeek-V3 style):  0.26 GB at S=4096  (5× further reduction)

  TRADE-OFF:
    Cache is smaller, but attention computation requires up-projecting c_KV
    to full K and V at each decode step (adds matrix-vector multiplies).
    Net effect: ~equivalent compute, significantly lower serving memory.

  RoPE COMPATIBILITY NOTE:
    RoPE encodes positions at the time K vectors are created.
    MLA caches position-agnostic c_KV latents plus a small k_rope component
    separately, preserving RoPE's relative-position property while enabling
    position-flexible caching.
```

2.6 Feed-Forward Networks — SwiGLU, Expansion Ratios, and FLOP Dominance

The FFN is the **dominant compute cost** in a transformer. It accounts for approximately 67% of all forward-pass FLOPs in a standard LLM. This is not intuitive — the name "attention" implies it is the important part. The math says otherwise.

2.6.1 Standard FFN and SwiGLU

```
STANDARD FFN (original Transformer, BERT):
  FFN(x) = W_2 · ReLU(W_1 · x + b_1) + b_2
  W_1 in R^(d_model x d_ff)   where d_ff = 4 × d_model  (4× expansion)
  W_2 in R^(d_ff x d_model)

  For d_model=8192: W_1 is 8192×32768, W_2 is 32768×8192
  Both parameters: 2 × 8192 × 32768 = 537M params per block

SWIGLU (LLaMA-2/3, PaLM, Gemma — used in every modern LLM):
  SwiGLU(x) = (W_1 · x  ⊙  SiLU(W_3 · x)) · W_2

  where SiLU(x) = x · sigmoid(x) = x / (1 + e^{-x})
  ⊙ = elementwise multiplication (the gating operation)

  THREE weight matrices: W_1, W_3 in R^(d_model x d_ff), W_2 in R^(d_ff x d_model)

  To match standard FFN parameter count, d_ff is adjusted:
    d_ff_swiglu = floor((2/3) × 4 × d_model / 256) × 256
                = floor(8/3 × d_model / 256) × 256
```

```
  LLaMA-3 70B: d_model=8192
    d_ff = floor(8/3 × 8192 / 256) × 256 = floor(21845.3 / 256) × 256
         = 85 × 256 = 21760... (LLaMA-3 uses 28672 — slightly different ratio)

  LLaMA-3 70B actual FFN config:
    W_1, W_3: 8192 × 28672 = 235M params each
    W_2:      28672 × 8192 = 235M params
    Total per block: 705M params in FFN

  COMPARE TO ATTENTION:
    GQA attention projections (Q+K+V+O): ~268M params per block
    FFN: 705M params per block  (2.6× more parameters than attention!)
    FFN FLOP fraction: ~67% of total (both compute and memory dominated by FFN)
```

2.6.2 Performance Regime of FFN Operations

```
FFN PERFORMANCE REGIME vs BATCH SIZE:

  For W_1: [d_model × d_ff] = [8192 × 28672]

  At batch=1 (decode, single token):
    Shape: [1 × 8192] × [8192 × 28672]
    FLOPs = 2 × 1 × 8192 × 28672 = 470M
    Bytes = 8192 × 28672 × 2B (BF16) = 470 MB (load weights) + tiny activations
    AI = 470M FLOP / 470 MB = ~1 FLOP/byte  -> DEEPLY memory-bound

  At batch=512 (training or large-batch serving):
    Shape: [512 × 8192] × [8192 × 28672]
    FLOPs = 2 × 512 × 8192 × 28672 = 241B
    Bytes = 470 MB (same weights!) + 512 × 8192 × 2B activations = 478 MB
    AI = 241B FLOP / 478 MB = ~504 FLOP/byte -> COMPUTE-BOUND (above ridge point)

  THE KEY INSIGHT: FFN weight matrices are fixed. At batch=1, you pay the full
  weight loading cost (470 MB) for almost zero compute. At batch=512, you amortize
  the weight loading across 512 tokens — same bytes, 512× more compute.
  This is the mathematical basis for why batch size is the primary serving lever
  (also shown for decode attention in Chapter 1, §1.3.3).
```

2.6.3 Why 4× Expansion? The Memory Hypothesis

```
THE FFN MEMORY HYPOTHESIS (Geva et al., 2021):
```

```
Observation: the FFN in a transformer behaves like a key-value memory store.
  Each row of W_1 is a "key pattern" — a detector for a specific input feature.
  Each column of W_2 is a "value" — what to add to the output if that key fires.
  The activation function (ReLU/SiLU) controls which keys activate.

With d_model=8192, d_ff=28672, L=80 layers:
  Total "memory slots" = 28672 × 80 = ~2.3M discrete patterns
  (Per-slot pattern: an 8192-dimensional feature vector in W_1)

This hypothesis explains:
  - Why scaling FFN dimension improves factual recall non-linearly
  - Why pruning random FFN neurons hurts more than expected
  - Why instruction fine-tuning works: it updates the key-value associations

Performance implication:
  The 4× expansion is not arbitrary. Reducing it saves FLOP and memory but
  measurably hurts model quality. GQA reduced KV heads with ~0% quality loss;
  reducing FFN width reliably does NOT have this property.
```

2.7 Positional Encoding — From Sinusoidal to RoPE

Attention is permutation-invariant: if you shuffle all tokens, the attention weights rearrange identically — the model sees a *set* of tokens, not a *sequence*. Positional encoding injects order information.

```
POSITIONAL ENCODING COMPARISON:

  Method            | Mechanism                       | Length Gen   | Used In

  Absolute          | Fixed sinusoid added to         | Poor — fails  | Original
  sinusoidal        | embedding at each position       | beyond train  |
Transformer 2017

  Learned           | Trainable position embedding    | None — hard   | GPT-2,
BERT
  absolute          | table                            | ceiling       |

  ALiBi             | Negative linear bias on         | Good — formula| BLOOM, MPT
                    | attention scores by distance     | works any len |

  RoPE              | Rotate Q,K vectors by           | Excellent —   | LLaMA-2/3,
                    | position-dependent angle         | 128K+ w/YaRN  | Mistral,
Gemini

ALiBi FORMULA:
  S_ij = (Q_i · K_j / sqrt(d_k)) - m_h · (i - j)
  where m_h = 2^{-8h/H} for head h of H total heads

  Effect: attending to a token 100 positions away penalized by 100 × m_h.
  Closer tokens preferred — recency bias built in. Zero extra parameters.
  Disadvantage: RoPE generally outperforms ALiBi on long-range reasoning tasks.
```

2.8 RoPE — Rotary Position Embedding: Full Derivation

RoPE is the dominant positional encoding in modern LLMs. It encodes position by rotating query and key vectors so that the dot product Q_i · K_j depends only on the *relative distance* (i−j), not absolute positions i and j independently.

2.8.1 The Core Idea: Relative Position via Rotation

```
ROPE — MATHEMATICAL FOUNDATION:
```

```
  Goal: make Q_i · K_j depend only on (i - j), not on i and j independently.

  Key mathematical property of 2D rotation matrices:
    R(alpha) · R(beta)^T = R(alpha - beta)

  If we rotate vectors: RoPE(q, pos) applies rotation R(pos × theta)
  Then:
    RoPE(q, i) · RoPE(k, j) = q_rotated · k_rotated
                            = f(i - j)    (depends only on relative distance)

  This is exactly what we want for KV caching:
    Cached K at position s, new Q at position t:
    dot product automatically encodes relative position (t - s)
    without recomputing K.

EXPLICIT COMPUTATION (for dimension pair (2t, 2t+1) of query vector q at position
pos):

  q_tilde_{2t}   = q_{2t}   × cos(pos × theta_t) - q_{2t+1} × sin(pos × theta_t)
  q_tilde_{2t+1} = q_{2t}   × sin(pos × theta_t) + q_{2t+1} × cos(pos × theta_t)

  where: theta_t = 1 / 10000^{2t/d_k}

  Different dimension pairs use different rotation frequencies:
    t=0:       theta_0 = 1.0              (fast rotation — short-range)
    t=d_k/4:   theta = 10000^{-0.5} ~ 0.01  (medium frequency)
    t=d_k/2-1: theta = 1/10000 = 0.0001  (slow rotation — long-range)

  The range of periods (1 to 10000) gives usable signal for positions 0..10000.

YARNS CONTEXT EXTENSION (Peng et al., 2023):
  Base theta=10000 limits effective context to ~8K tokens.
  LLaMA-3 uses theta_base=500000 — extending natural context to ~128K.
  YaRN modifies theta_t further for positions beyond base:
    theta_t^{YaRN} = 1 / (alpha × base^{2t/d_k})
    where alpha is a scale factor tuned for the target context length.

  This allows extending LLaMA-3 from 8K (training context) to 128K
  with minor fine-tuning (LoRA, a few thousand gradient steps).
```

2.8.2 Why RoPE is KV Cache Compatible

```
ROPE + KV CACHE COMPATIBILITY — CRITICAL PROPERTY:
```

```
  Standard challenge: position encoding must be "baked in" to K and V vectors
  at cache time. When a new Q at position t attends to cached K at position s:

    For absolute PE: K_s = K_base + PE(s)  — position already added
                     At decode time, Q_t = Q_base + PE(t)
                     Q_t · K_s includes cross-terms PE(t) · PE(s) + K_base · PE(t)
+ ...
                     This DOES NOT simplify to a function of (t-s). ❌

    For RoPE:        K_s = RoPE(K_base, s)
                     Q_t = RoPE(Q_base, t)
                     Q_t · K_s = RoPE(Q_base, t) · RoPE(K_base, s) = f(t-s)  ✓

  Therefore: cached RoPE-encoded K vectors from any past position can be
  used directly with any future Q — no recomputation needed.
  This is the mathematical reason RoPE is the universal standard for all
  KV-cache-dependent LLMs.

IMPLEMENTATION NOTE:
  RoPE is applied inside the attention kernel, after QKV projections but before
  the QK^T dot product. In FlashAttention implementations, RoPE is fused directly
  into the attention kernel — the cos/sin tables are precomputed and applied via
  elementwise multiply + in-place rotate at negligible overhead.
```

2.9 Normalization — LayerNorm, RMSNorm, Pre-Norm vs Post-Norm

This section contains the complete normalization reference for AI/ML performance engineers, including the BatchNorm failure proof that appears in nearly every principal-level interview.

2.9.1 The Three Normalization Variants

```
NORMALIZATION COMPARISON — COMPLETE REFERENCE:

                    BatchNorm         LayerNorm         RMSNorm
  ──────────────────────────────────────────────────────────────────────
  Formula:          y = (x-mu_B)/sigma_B  y = (x-mu_L)/sigma_L  y = x/RMS(x)
                    × gamma + beta       × gamma + beta          × gamma

  Normalizes over:  Batch + spatial dims  Feature dim            Feature dim
                    (per feature)         (per sample)            (per sample)
```

```
  Works at B=1?     NO (variance=0)       YES                     YES

  Parameters:       2D (gamma, beta)      2D (gamma, beta)        1D (gamma only)
                                                                   no bias term

  FLOPs per token:  ~5D (2 reductions)   ~5D (2 reductions)     ~3D (1 reduction)
                    (mean + variance)     (mean + variance)      (sum of squares)

  Arithmetic        ~1.3 FLOP/byte        ~1.3 FLOP/byte         ~1.0 FLOP/byte
  intensity:
  -> ALL THREE ARE DEEPLY MEMORY-BANDWIDTH BOUND (H100 ridge = 295 FLOP/byte)
  -> ALL THREE MUST BE FUSED with adjacent GEMM to avoid HBM round-trip

  Speed vs           —                    baseline               ~15-30% faster
  LayerNorm:                                                      (1 reduction vs
2)

  Used in:          ResNet, CNNs,         Original Transformer,  LLaMA 1/2/3,
                    Vision models         BERT, GPT-2            Mistral, Gemma,
                    (NEVER in LLMs)                              Falcon, PaLM

  Fusion priority:  N/A (not in LLMs)    HIGH — fuse with       HIGH — same
                                          adjacent GEMM           as LayerNorm
  ──────────────────────────────────────────────────────────────────────────
```

2.9.2 BatchNorm Failure at Batch Size 1 — The Proof

This is one of the most commonly asked normalization questions at principal-level interviews.

```
BATCHNORM DEFINITION:
  y_i = (x_i - mu_batch) / sigma_batch × gamma + beta

  where:
    mu_batch    = (1/B) × sum_j(x_j)              (mean over batch dimension)
    sigma_batch = sqrt((1/B) × sum_j(x_j - mu)^2) (std over batch dimension)

FAILURE AT B=1:
  At B=1:
    mu_batch = x_1  (the only sample is its own mean)

    x_i - mu_batch = x_1 - x_1 = 0   (for every element)
    sigma_batch = sqrt(0) = 0

  With epsilon stabilization:
    y_i = 0 / sqrt(0 + epsilon) × gamma + beta
        = 0 × gamma + beta
        = beta                          (CONSTANT — input entirely discarded!)

  Result: At B=1, BatchNorm output is a fixed constant (beta), regardless
  of input values. All information in x is destroyed.

WHY B=1 IS UNAVOIDABLE IN LLM INFERENCE:
  During autoregressive decode, each token generation step operates on
  one token at a time in the sequence dimension. The "effective batch" for
  the normalization operation is B=1 per sequence.

  More precisely:
    Prefill: B=batch_size, S=prompt_length  -> BatchNorm would work
    Decode:  effectively S=1 per step for each sequence in the batch
             -> BatchNorm produces constant output -> information loss

LAYERNORM DOES NOT HAVE THIS PROBLEM:
  y_i = (x_i - mu_layer) / sigma_layer × gamma_i + beta_i

  At B=1, d_model=8192:
    mu_layer    = mean over 8192 features of the single sample  (well-defined)
    sigma_layer = std over 8192 features of the single sample   (well-defined)

  8192 values provides more than enough statistical stability.
  LayerNorm output is fully input-dependent at any batch size including B=1.

INTERVIEW ONE-LINER:
```

```
  "BatchNorm normalizes across the batch dimension — at batch=1 the sample is
  its own mean, variance is zero, and the output is a constant bias.
  LayerNorm normalizes across the feature dimension — well-defined at any batch
  size. Autoregressive decode IS batch=1, so BatchNorm is architecturally
  incompatible with LLM inference."
```

2.9.3 The Four Rules Every Performance Engineer Must Know

```
NORMALIZATION — FOUR RULES (memorize these):

  1. RMSNorm saves ~15-30% compute vs LayerNorm at equal quality.
     This is why it replaced LayerNorm in all post-2022 production models.
     The mean-centering step in LayerNorm is empirically unnecessary for LLMs.

  2. Pre-Norm provides guaranteed gradient flow through residuals.
     The +1 in dy/dx = 1 + dF/dx is what enables 80-layer models to train.
     This is why all modern LLMs dropped Post-Norm after GPT-2.

  3. ALL norm variants are memory-bound (AI ~1-5 FLOP/byte).
     They are always listed as fusion candidates in kernel optimization.
     An un-fused RMSNorm reads + writes the full activation tensor to HBM.
     At LLaMA-3 70B scale: 160 un-fused norms per step = ~21 GB avoidable traffic.

  4. BatchNorm is mathematically undefined at B=1.
     This is the definitive reason it cannot appear in autoregressive LLMs.
     You will never see BatchNorm in a production language model.
```

2.9.4 The Un-Fused Normalization Cost — Why Fusion Is Mandatory

```
UN-FUSED NORMALIZATION HBM COST (LLaMA-3 70B):

  Each RMSNorm call reads + writes the activation tensor:
    Bytes per call = 2 (read + write) × S × d_model × 2 (BF16)
                   = 2 × 4096 × 8192 × 2 = 134 MB per RMSNorm call

  LLaMA-3 70B has 2 RMSNorm per block × 80 blocks = 160 RMSNorm calls per step
    Total un-fused HBM traffic = 160 × 134 MB = 21.4 GB avoidable per step

  At H100 HBM bandwidth (3.35 TB/s):
    Un-fused overhead = 21.4 GB / 3.35 TB/s = 6.4 ms per step (wasted)

  At 80 ms total step time (reasonable for medium batch):
    RMSNorm overhead = 6.4 / 80 = 8% of step time — purely avoidable!
```

```
  Fused: RMSNorm + GEMM executes as one kernel. Activations stay in registers
  or L1/SMEM. Zero extra HBM traffic. This is the canonical Chapter 9 example.
```

2.10 The Complete FLOP Budget — LLaMA-3 70B Worked Example

Every layer in a transformer has a computable FLOP cost. This section derives the full budget from first principles for LLaMA-3 70B — the reference model used throughout this book.

2.10.1 Model Configuration

```
LLAMA-3 70B CONFIGURATION: [SHIPPED]

  d_model         = 8,192    (hidden dimension)
  d_head          = 128      (attention head dimension)
  H_q             = 64       (query heads)
  H_kv            = 8        (KV heads — GQA)
  d_ffn           = 28,672   (FFN intermediate dimension)
  L               = 80       (transformer layers)
  Vocabulary size = 128,256  (token vocabulary)
  Training context= 8,192    (pretraining context length)
  Max context     = 128,000  (with RoPE extension)
```

2.10.2 Per-Operation FLOPs — One Layer, One Token

```
FLOP ACCOUNTING — ONE TRANSFORMER LAYER, ONE TOKEN (S=1):

  NOTATION: FLOPs for GEMM with shapes [m,k] x [k,n] = 2 × m × k × n

  1. QKV PROJECTIONS:
     Q projection: [1 × 8192] × [8192 × 8192]    = 2 × 8192 × 8192
                                                  = 134,217,728 ≈ 134M FLOPs
     K projection: [1 × 8192] × [8192 × 1024]    = 2 × 8192 × 1024
                                                  = 16,777,216 ≈ 16.8M FLOPs
     V projection: same as K                      = 16.8M FLOPs
     Total QKV: 134.2 + 16.8 + 16.8              = 167.8M FLOPs/token

  2. ATTENTION SCORE COMPUTATION (S=1 decode, attending to context length C):
     Q×K^T: [1 × 128] × [C × 128]^T = 2 × 128 × C per head × 64 heads
           = 16,384 × C FLOPs  (scales with context C)
```

```
      Softmax: O(C) per head  (negligible vs GEMM)
      A×V:   [1 × C] × [C × 128] = 2 × C × 128 per head × 64 heads
            = 16,384 × C FLOPs
      Total attention: 32,768 × C FLOPs
      At C=4096: 32,768 × 4096 = 134.2M FLOPs
      At C=128K: 32,768 × 128K = 4.3B FLOPs  (attention dominates at long context!)

   3. ATTENTION OUTPUT PROJECTION:
      [1 × 8192] × [8192 × 8192] = 2 × 8192 × 8192 = 134.2M FLOPs

   4. FFN — THREE GEMMS (SwiGLU):
      Gate (W_3): [1 × 8192] × [8192 × 28672]  = 2 × 8192 × 28672 = 469.8M FLOPs
      Up   (W_1): same                          = 469.8M FLOPs
      SiLU elementwise: 28672                   = ~0.03M FLOPs (negligible)
      Down (W_2): [1 × 28672] × [28672 × 8192] = 2 × 28672 × 8192 = 469.8M FLOPs
      Total FFN: 3 × 469.8M                    = 1,409.5M ≈ 1.41B FLOPs

   5. NORMALIZATION (2 × RMSNorm):
      Each: ~3 × d_model = 3 × 8192 ≈ 24.6K FLOPs
      Total: 49K FLOPs  (0.003% of layer — truly negligible)

   PER-LAYER SUMMARY (S=1, C=4096):
     QKV projections:           167.8M  (9.8%)
     Attention score + output:  268.4M  (15.7%)
     FFN (SwiGLU):            1,409.5M  (82.4%)
     Normalization:               0.1M  (0.0%)
     TOTAL PER LAYER:         1,845.8M ≈ 1.85B FLOPs

   LAYER FLOP DISTRIBUTION:
     FFN:           ~76% of layer FLOPs    (dominant!)
     Attention:     ~24% of layer FLOPs
     Everything else: < 1%
```

2.10.3 Full Model FLOP Budget

```
COMPLETE MODEL FLOP BUDGET — LLAMA-3 70B:

  Per-layer FLOPs (short context, per token): ~1.85B FLOPs
  80 layers: 80 × 1.85B = 148B FLOPs per token

  APPROXIMATION FORMULA (widely used):
    FLOPs per token ≈ 2 × N_params
    2 × 70B = 140B FLOPs per token  (within 6% of exact calculation above)
```

```
    This works because: in a dense transformer, the total parameter count
    is dominated by projection matrices. Each param is used in one multiply
    and one accumulate = 2 FLOPs per parameter per token.

  TRAINING FLOP FORMULA:
    Training FLOPs ≈ 6 × N_params × D_tokens
    Factor of 6 = forward (2N) + backward (4N, because backward ~2× forward)

    LLaMA-3 70B trained on 15T tokens:
      6 × 70B × 15T = 6.3 × 10^24 FLOPs
      = 6.3 ZettaFLOPs

  GPU-HOURS TO TRAIN LLaMA-3 70B ON H100:
    At 40% MFU (typical large-scale training), sustained FLOP/s per H100:
      0.40 × 989 × 10^12 = 395.6 × 10^12 FLOP/s

    On 6,144 H100s (Meta's reported configuration):
      Total cluster FLOP/s = 6,144 × 395.6 × 10^12 = 2.43 × 10^18 FLOP/s

    Training duration:
      6.3 × 10^24 / 2.43 × 10^18 = 2,593,000 seconds ≈ 30 days  [ESTIMATED]
      (Consistent with Meta's reported training timeline)

  THE FIVE NUMBERS EVERY AI PERF ENGINEER MUST MEMORIZE:
    #1  FLOPs/token (inference)  = 2 × N_params         (e.g., 140B for 70B model)
    #2  Bytes (BF16 inference)   = 2 bytes/param         (e.g., 140 GB for 70B)
    #3  Bytes (training state)   = 18 bytes/param        (e.g., 1.26 TB for 70B)
        (BF16 weights: 2, FP32 weights: 4, FP32 grad: 4, FP32 Adam m+v: 4+4 = 18)
    #4  Training FLOPs           = 6 × N × D_tokens
    #5  Chinchilla-optimal D     ≈ 20 × N_params
```

2.10.4 Memory Breakdown — LLaMA-3 70B

```
COMPLETE MEMORY BREAKDOWN — LLAMA-3 70B:

  INFERENCE (BF16, single sequence):
    Model weights:  70B × 2 bytes          = 140 GB   (minimum GPU memory
required)
    KV cache (GQA, BF16, S=4096):
      2 × 4096 × 8 × 128 × 2B × 80 layers = 1.34 GB  per sequence
    Activations:    ~67 MB (one block at a time, no checkpointing)
    Total inference: ~141.4 GB per sequence at S=4096
```

```
  SERVING 8 concurrent sequences (S=4096):
    Weights (shared): 140 GB
    KV cache × 8:     8 × 1.34 GB = 10.7 GB
    Total:            ~150.7 GB   <- fits on two H100 SXM (2 × 80 GB = 160 GB)

  TRAINING (BF16 weights + FP32 optimizer state, ZeRO-1):
    Model weights (BF16):    70B × 2B         = 140 GB
    Gradients (BF16):        70B × 2B         = 140 GB
    FP32 master weights:     70B × 4B         = 280 GB
    Adam m (FP32):           70B × 4B         = 280 GB
    Adam v (FP32):           70B × 4B         = 280 GB
    Total training state:    70B × 16B        = 1,120 GB = 1.12 TB

    Activation memory per block (no checkpointing, S=4096, batch=1):
      ~4096 × 8192 × 2B × ~16 tensors per block ≈ 1 GB per block
      80 blocks: ~80 GB additional (with activation checkpointing: ~5 GB)

    Per GPU with ZeRO-3 (across 8 GPUs): 1.12 TB / 8 = 140 GB — just fits H100!
    At 64 GPUs (DP=8, ZeRO-3): 1.12 TB / 64 = 17.5 GB model state per GPU +
activations
```

2.11 Encoder, Decoder, and Encoder-Decoder Architectures

```
ARCHITECTURE VARIANTS:

  Architecture    | Attention         | Masking        | Use Case           |
Examples
```

```
  Encoder-only    | Bidirectional     | None (sees all)| Classification,    |
BERT, RoBERTa,
                  | (each token sees  |                | embeddings,        |
DeBERTa
                  | all other tokens) |                | retrieval          |
```

```
  Decoder-only    | Causal (each      | Lower-triangle | Text generation,   | GPT,
LLaMA,
                  | token sees past   | mask           | completion,        |
Mistral, Gemma,
```

```
                  | tokens only)      |                | all frontier LLMs  |
Claude, GPT-4
```

```
  Encoder-Decoder | Encoder: bidir    | Causal in      | Translation,       | T5,
BART,
                  | Decoder: causal   | decoder only   | summarization,     | mT5,
                  | + cross-attention |                | instruction-follow |
Whisper (audio)

WHY DECODER-ONLY WON FOR LLMs:
  1. Simple training objective: next-token prediction on any unlabeled text.
     No need for paired input-output data (unlike encoder-decoder).
  2. Scales cleanly with compute — scaling laws hold predictably.
  3. Performs instruction-following after RLHF fine-tuning.
  4. KV cache is natural: past tokens are cached, new tokens attend to all past.

  Every frontier model as of 2026 (GPT-4, LLaMA-3, Claude, Gemini, Mistral)
  is decoder-only.

INFERENCE PROFILE COMPARISON:
  Encoder-only:   One forward pass per input. No autoregressive decode. No KV
cache.
                  Throughput: very high (batch all tokens together).

  Decoder-only:   Autoregressive — one token generated per step.
                  Prefill (process prompt): compute-bound GEMM.
                  Decode (generate): memory-bandwidth-bound, one token at a time.
                  KV cache grows with context length.

  Encoder-Decoder: Encode prompt once (bidirectional, fast).
                   Decode output autoregressively (same as decoder-only).
                   Cross-attention: Q from decoder, K/V from encoder (computed
once, cached).
```

2.12 Scaling Laws — What Performance Engineers Must Know

Scaling laws describe how model quality improves as a function of parameters, training data, and compute budget. They drive every infrastructure investment decision at major labs.

```
SCALING LAW FUNDAMENTALS:

  Kaplan et al. (2020) — Power-law relationships:
    Loss ~ A × N^{-alpha_N}  where alpha_N ~ 0.076  (quality vs parameters)
    Loss ~ B × D^{-alpha_D}  where alpha_D ~ 0.095  (quality vs training data)

    For fixed compute budget C ≈ 6ND (FLOPs):
    Kaplan (2020) recommendation: scale N much faster than D.
    This turned out to be wrong — Chinchilla corrected it.

  Chinchilla (Hoffmann et al., 2022) — Revised compute-optimal:
    For fixed compute budget C:
      N_opt ≈ (C/6)^{0.5} × 0.5  (parameters)
      D_opt ≈ 20 × N_opt           (training tokens)

    "Chinchilla rule": optimal training uses ~20 tokens per parameter.
    A 70B model is compute-optimal when trained on ~1.4T tokens.

  PRODUCTION PRACTICE: DELIBERATE OVER-TRAINING:
    LLaMA-3 8B:   trained on 15T tokens  (Chinchilla optimal: ~160B = 94× over-
trained)
    LLaMA-3 70B:  trained on 15T tokens  (Chinchilla optimal: ~1.4T = 11× over-
trained)

    WHY? Inference cost scales with N, not D.
    A smaller model trained on more data has LOWER serving cost at equal quality.

    The math: serving 10M users per day for 1 year at 1,000 tokens/request:
      3.65T tokens served. At $0.001/1K tokens:
      7B model: $3.65M  vs  70B model: $36.5M  (10× cost difference)
      The extra training cost of $1M more data pays back in one week of serving.

  THE CHINCHILLA INSIGHT FOR PERF ENGINEERS:
    "Compute-optimal" from training perspective ≠ "cost-optimal" for the business.
    The model that minimizes training FLOPs is almost never the model that
minimizes
    serving cost. This is why distillation, pruning, and quantization are
critical.
```

2.12.1 Compute Budget → Hardware Fleet Sizing

```
TRAINING DURATION FORMULA:
```

```
  Time_seconds = Total_FLOPs / (N_GPUs × GPU_peak_FLOPS × MFU)
  Time_days    = Time_seconds / 86400

WORKED EXAMPLE — LLaMA-3 70B:
  Total FLOPs = 6 × 70B × 15T = 6.3 × 10^24 FLOPs
  Hardware: 6,144 × H100 SXM5 (989 TFLOPS BF16)
  MFU: 43% (typical large-scale)

  Sustained FLOP/s = 6,144 × 989 × 10^12 × 0.43 = 2.61 × 10^18 FLOP/s
  Duration = 6.3 × 10^24 / 2.61 × 10^18 = 2,414,942 s ≈ 27.9 days  [ESTIMATED]

FLEET SIZING FOR TARGET TRAINING DURATION:
  N_GPUs = Total_FLOPs / (Target_days × 86400 × GPU_peak_FLOPS × MFU)

  For 30-day target, H100, 43% MFU:
    N_GPUs = 6.3 × 10^24 / (30 × 86400 × 989 × 10^12 × 0.43)
           = 6.3 × 10^24 / 1.107 × 10^21
           = 5,690 GPUs  ->  round up to 6,144 (nearest power-of-2 × 8-node)
```

2.13 Modern Architecture Comparison

```
MODERN LLM ARCHITECTURE REFERENCE:

  Model           | Active Params | Layers | d_model | Attention    | FFN     |
Context
-----------------------------------------------------------------------------------
  LLaMA-3 8B      | 8B            |  32    | 4,096   | GQA 32Q/8K   | SwiGLU  |
128K
  LLaMA-3 70B     | 70B           |  80    | 8,192   | GQA 64Q/8K   | SwiGLU  |
128K
  LLaMA-3 405B    | 405B          | 126    | 16,384  | GQA 128Q/16K | SwiGLU  |
128K
  Mistral 7B      | 7.3B          |  32    | 4,096   | GQA + SWA    | SwiGLU  |
32K
  Mixtral 8×7B    | 46.7B/12.9B*  |  32    | 4,096   | GQA          | MoE top-2|
32K
  DeepSeek-V3     | 671B/37B*     |  61    | 7,168   | MLA          | MoE 256E |
128K
  Gemma 2 27B     | 27B           |  46    | 4,608   | GQA          | GeGLU   | 8K
```

```
  Qwen2.5 72B     | 72B           |  80    | 8,192   | GQA          | SwiGLU  |
128K

  * MoE: (Total params / Active params)   SWA: Sliding Window Attention

KEY ARCHITECTURAL DECISIONS AND THEIR PERFORMANCE IMPLICATIONS:

  Decision 1 — GQA vs MHA:
    All modern production models use GQA.
    Impact: 8× KV cache reduction at GQA-8 vs MHA-64.
    Enables 128K+ context on practical hardware.
    Quality cost: ~0% with proper training.

  Decision 2 — Dense vs MoE:
    MoE decouples model quality from inference compute.
    DeepSeek-V3: 671B total params, only 37B active per token.
    GPT-4-class quality at ~1/10th training compute vs equivalent dense model.
    Infrastructure cost: All-to-All communication (see Ch 13), expert placement.

  Decision 3 — RoPE base theta:
    theta=10000 (GPT-2 era): ~4K effective context
    theta=500000 (LLaMA-3): ~128K effective context
    YaRN: extends to 1M+ with fine-tuning
    Impact: zero inference overhead, enables long-context applications.

  Decision 4 — SwiGLU vs alternatives:
    Universal adoption of SwiGLU/GeGLU in all post-2022 models.
    Cost: 3 weight matrices instead of 2 (+50% FFN parameters vs same d_ffn).
    Quality: +0.5-1% on standard benchmarks. Adopted by every major lab.

  Decision 5 — Pre-Norm placement:
    Universal adoption since GPT-2. No modern frontier model uses Post-Norm.
    Performance: allows deeper models (80-126 layers) with stable training.
    Zero inference overhead (just changes computation order).
```

2.14 The Performance Engineering Map of the Transformer

Every operation in the transformer falls into one of two bottleneck regimes. This map is the workload specification that drives Chapters 6–9.

TRANSFORMER PERFORMANCE ENGINEERING MAP:

OPERATION	REGIME	AI	PRIMARY OPTIMIZATION
FFN GEMM (training, large batch)	COMPUTE-BND	~300-500 FLOP/byte	FP8, larger tiles, Tensor Core alignment
Prefill attention (long sequences)	NEAR RIDGE	~200 FLOP/byte	FlashAttention (IO-aware tiling), chunked prefill
FFN GEMM (decode, small batch)	MEMORY-BND	~1-5 FLOP/byte	Weight quantization (INT4 -> halve bytes loaded)
Decode attention (batch=1)	MEMORY-BND	~1 FLOP/byte	KV compression, GQA, MQA, prefix caching
RMSNorm / LayerNorm	MEMORY-BND	~1-5 FLOP/byte	Fuse with adjacent GEMM (eliminate HBM round-trip)
RoPE application	MEMORY-BND	~2 FLOP/byte	Fuse into FlashAttention kernel (standard practice)
Embedding lookup	MEMORY-BND	~1 FLOP/byte	Limit to fp16/bf16, compress vocab if needed
Softmax	MEMORY-BND	~3 FLOP/byte	Fuse into FlashAttention or online softmax

SUMMARY:

The ONLY compute-bound operation is large-batch GEMM (training, prefill at scale).

EVERYTHING else in a transformer is memory-bandwidth-bound.

```
    The principal engineer who knows this map can predict the impact of every
    proposed optimization before running a single experiment.
```

2.15 Key Takeaways and Review Questions

```
FIVE NUMBERS TO MEMORIZE (LLaMA-3 70B as reference):
  1. FLOPs/token (inference)  = ~140B    (≈ 2 × N_params)
  2. Model size (BF16)        = 140 GB   (= 2 bytes × N_params)
  3. Training state           = 1.12 TB  (= 16 bytes × N_params with AdamW)
  4. KV cache/seq (GQA, 4K)   = 1.34 GB  (2 × S × H_kv × d_head × 2B × L)
  5. Training FLOPs           = 6.3×10^24 (= 6 × 70B × 15T tokens)

ATTENTION COMPLEXITY:
  QKV projections: O(S) — linear in sequence length
  Attention scores (QK^T + AV): O(S^2) — quadratic
  Attention dominates over projections when S > 1.5 × d_model
  For LLaMA-3 70B: crossover at S ~ 12K tokens

ARCHITECTURE DECISIONS AND THEIR PERFORMANCE RATIONALE:
  GQA (not MHA):       8× KV cache reduction, ~0% quality loss, enables 128K
context
  RMSNorm (not LN):    ~20% faster, equal quality, 1D params vs 2D
  Pre-Norm (not Post): stable training at 80+ layers, no warmup needed
  SwiGLU (not ReLU):   +0.5-1% quality, 3 weight matrices (50% more FFN params)
  RoPE (not sinusoidal): KV-cache compatible, extensible to 128K+ with YaRN

SCALING LAW CORE INSIGHT:
  Chinchilla-optimal: D ≈ 20 × N (e.g., 1.4T tokens for 70B model)
  Modern practice: D >> Chinchilla-optimal (LLaMA-3: 15T for 70B = 11× over)
  Reason: serving cost >> training cost at production scale.
  Compute-optimal training != cost-optimal production deployment.

BATCHNORM FAILURE:
  "BatchNorm outputs a constant at B=1. LLM decode is B=1. Never use BatchNorm in
LLMs."
```

Review Questions

Difficulty: Foundational

Question 1 (FLOP Counting): A transformer model has d_model=4096, 32 layers, 32 query heads, 8 KV heads (GQA), d_head=128, d_ffn=11008 (SwiGLU). (a) How many parameters are in

one attention block? One FFN block? (b) What is the approximate total FLOPs per token for inference? (c) What is the model size in GB for BF16 inference? What is the training state size?

Expected answer: (a) Attention: Q=4096×4096=16.8M, K=4096×512=2.1M, V=2.1M, O=4096×4096=16.8M → 37.8M per block. FFN: 3 × 4096×11008 = 3 × 45.1M = 135.3M per block. (b) Total params ≈ 32 × (37.8M + 135.3M) = 32 × 173.1M = 5.54B → FLOPs/token ≈ 2 × 5.54B = 11.1B. (c) Size = 5.54B × 2B = 11.1 GB. Training state = 5.54B × 16B = 88.6 GB.

Difficulty: Intermediate

Question 2 (Attention Variants): You are building a serving system for a 70B model (LLaMA-3 70B config: 80 layers, 64 query heads, d_head=128, BF16). Compare KV cache memory at S=8192 context for MHA, GQA-8, and MQA. Which enables fitting 4 concurrent sequences on a single H100 (80 GB total, ~140 GB for weights)?

Expected answer: Per-layer KV = 2 × S × H_kv × d_head × 2B. MHA (H_kv=64): 2×8192×64×128×2 = 268M bytes/layer × 80 = 21.5 GB per sequence. 4 sequences = 86 GB + 140 GB weights = impossible. GQA-8 (H_kv=8): 2×8192×8×128×2 = 33.6M bytes/layer × 80 = 2.7 GB per sequence. 4 sequences = 10.7 GB + 140 GB = 150.7 GB — still too large for one H100, needs two. MQA (H_kv=1): 2×8192×1×128×2 = 4.2M bytes/layer × 80 = 335 MB per sequence. 4 sequences = 1.3 GB + 140 GB = 141.3 GB — fits on two H100s (160 GB), nearly fits one with quantization. Conclusion: GQA-8 is the practical solution — LLaMA-3 uses this exact config. It enables 128K context while keeping KV cache manageable.

Difficulty: Principal Level

Question 3 (Normalization Architecture Interview): A new intern proposes replacing RMSNorm with BatchNorm in your LLM serving system, arguing that "BatchNorm is faster and widely used in CV." Write a complete technical rebuttal covering: (a) why BatchNorm fails at decode time mathematically, (b) whether RMSNorm is actually faster than LayerNorm and by how much, (c) what happens to un-fused RMSNorm at LLaMA-3 70B scale.

Expected answer: (a) BatchNorm normalizes across the batch dimension. At B=1 (every decode step), the single sample is its own mean → x_i - mu = 0 → output = constant bias. All input information discarded. Mathematically provable, not a configuration issue. (b) RMSNorm eliminates the mean-centering reduction (one fewer reduction pass). In practice: ~15-30% faster kernel execution. Empirically equal or better quality vs LayerNorm on all major benchmarks (used in LLaMA, Mistral, Gemma, Qwen). Parameters: d vs 2d (no bias term). (c) Un-fused RMSNorm at LLaMA-3 70B: 160 calls per step (2 per layer × 80 layers), each read+write 4096×8192×2B = 134 MB. Total: 160 × 134 MB = 21.4 GB avoidable HBM traffic per step. At 3.35 TB/s HBM bandwidth: 6.4 ms wasted per step — up to 8% of step time purely due to un-fused norms. The correct fix is kernel fusion (Chapter 9), not changing normalization type.

Difficulty: Principal Level

Question 4 (Scaling Laws Applied): Your company is deciding between training a 7B model on 150B tokens vs a 70B model on 15B tokens, with identical total compute budget (both use 6ND = 6 × 7B × 150B = 6.3×10^21 FLOPs). The 7B model will be deployed to serve 1M users/day at 500 output tokens/request for 2 years. Assuming equal quality (Chinchilla suggests these are roughly equivalent), make the infrastructure argument for which model to choose.

Expected answer: The models have equal quality by assumption (Chinchilla-optimal allocation). The decision is entirely about serving economics. Serving cost is dominated by model size (inference FLOPs/token = 2×N): 7B model: 14B FLOPs/token → 7× cheaper per token than 70B. Daily tokens served: 1M users × 500 tokens = 500M tokens/day. Over 2 years: 365 × 500M = 182.5B tokens. At H100 (24 tokens/sec/GPU for 70B, 168 tokens/sec for 7B [ESTIMATED]): 70B: needs ~70 H100s for 1M users/day. 7B: needs ~10 H100s. Hardware cost savings over 2 years: 60 GPUs × 730 days × $3/hr × 24hr = $3.15M saved. Training cost difference is ~zero (same compute budget). Decision: choose 7B. The principle: inference runs for years; training runs once. The model that minimizes serving FLOP/token wins at production scale, which is exactly why LLaMA-3 8B was trained on 15T tokens (11× Chinchilla-optimal) rather than training a 70B model on the same compute budget.

Further Reading

1. *Attention Is All You Need* — Vaswani et al. (2017). `arxiv.org/abs/1706.03762` The original transformer paper. Read Sections 1-4 carefully. Derive the FLOPs.
2. *RoFormer: Enhanced Transformer with Rotary Position Embedding* — Su et al. (2021). `arxiv.org/abs/2104.09864` The RoPE paper. Read the relative position proof.
3. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints* — Ainslie et al. (2023). `arxiv.org/abs/2305.13245` The GQA paper. Read the KV cache memory analysis.
4. *Training Compute-Optimal Large Language Models* — Hoffmann et al. (2022). `arxiv.org/abs/2203.15556` The "Chinchilla" paper. Core reading for cluster sizing.
5. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* — Dao et al. (2022). `arxiv.org/abs/2205.14135` The IO complexity analysis of standard vs tiled attention is the best applied example of roofline-based algorithm design in the literature.
6. *DeepSeek-V3 Technical Report* — DeepSeek AI (2024). `arxiv.org/abs/2412.19437` MLA architecture, MoE at 671B scale, FP8 training. The most important recent architecture paper for systems engineers.

— End of Chapter 2 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved